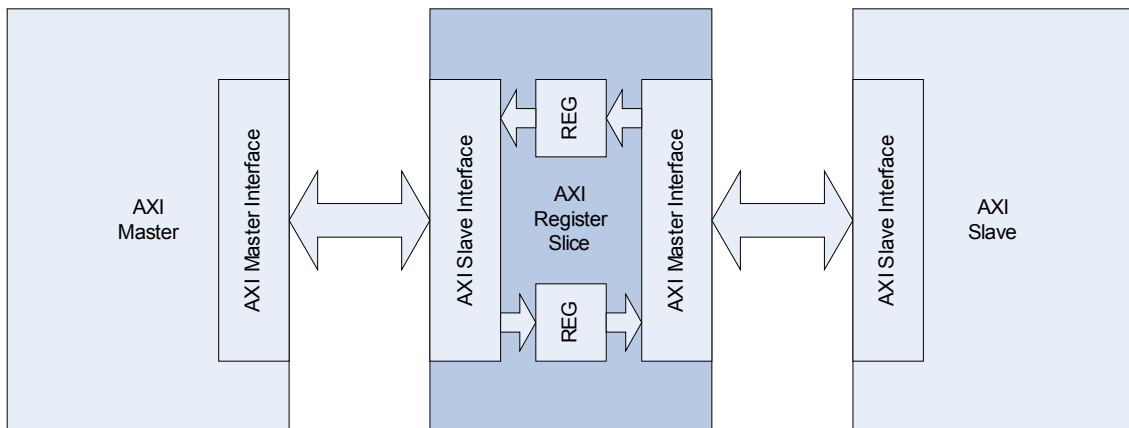


Title	AXI Register Slice Integration Guide
Project	None
Description	AXI Register Slice Integration Guide
Author	holelee
Date	23/May/2005

(*) Register Slice의 사용

AXI Register Slice는 AXI master가 AXI slave로 연결될 때 중간에 사용함으로써 전달되는 정보를 register로 latch하여 master와 slave 사이에 timing을 독립시켜주는 module이다. AXI master와 AXI slave가 서로 멀리 떨어진 경우에 대해서는 routing delay로 인해 동작 clock을 높이는데 한계가 생길 수 있다. 이런 경우 그 사이에 register slice를 아래 그림과 같이 중간에 삽입하여 사용하게 되면 AXI master와 AXI register slice, AXI register slice와 AXI slave 사이의 timing이 분리가 되므로 그로 인해 전체적인 동작 clock는 높아질 수 있다. 물론 이렇게 register slice를 사용하게 되면 정보 전달에 1 cycle의 delay가 추가로 발생하게 되지만, interface의 bandwidth에는 영향을 주지 않으므로 특별히 문제가 되지 않는다.



Register Slice의 연결

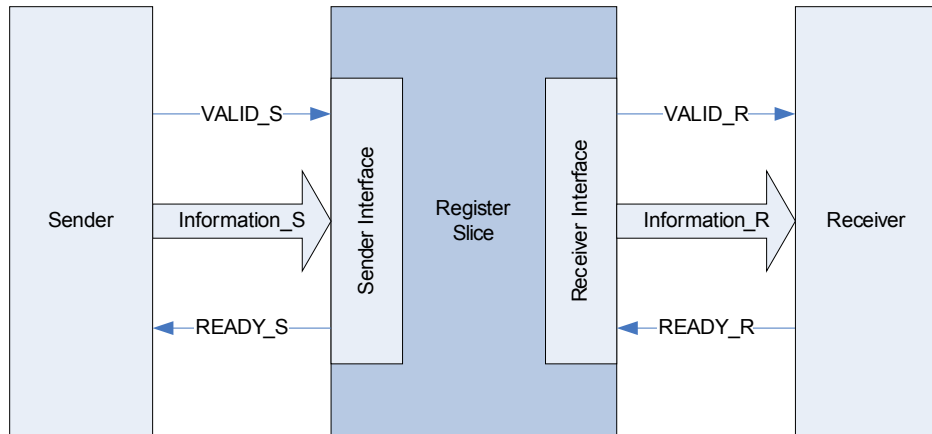
AXI BUS에서는 AXI BUS의 port(AXI master와 AXI slave를 연결하는 port)에 사용할 수 있고, 특정 Slave나 Master가 silicon상에서 너무 멀리 떨어져 routing delay가 발생한 경우에는 그 사이에도 사용할 수 있다.

(*) Register Slice의 형태

기본적으로 AXI는 5개의 channel로 구성되어 있고 각 channel은 독립적으로 동작하도록 되어 있다. 또한 모든 channel의 handshake는 VALID/READY를 사용하여 동일한 방법으로 연결된다. 다만 각 channel을 통해서 전달되는 정보의 양(즉 bit-width)만이 차이가 난다. 따라서 Register Slice를 AXI interface 전체에 대해서 설계할 필요는 없고, 단일 채널에 대해서 bit-width를 configuration할 수 있는 module로 설계하면 그것을 5번 instantiation하여 전체 AXI interface에 사용할 수 있게 된다.

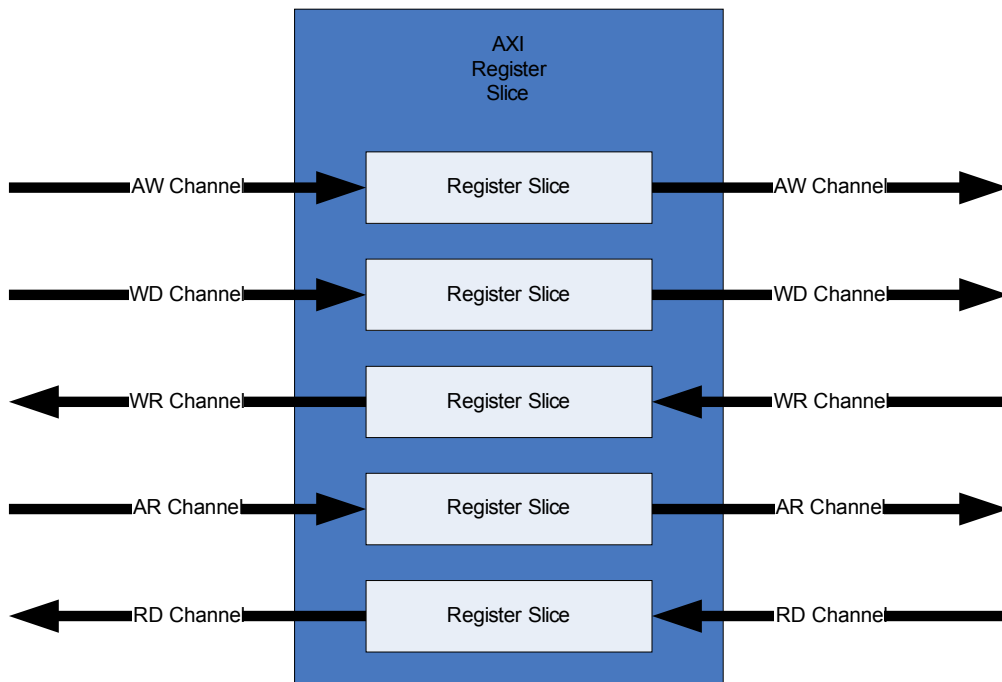
아래 그림이 단일 채널에 대한 Register Slice의 형태이다. 기본적으로 접미어 _S는 Sender를

의미하며 _R은 Receiver를 의미한다. Sender가 보내주는 Information과 VALID를 Registering하여 Receiver쪽으로 전달하는 일을 하게 된다. READY의 경우 registering하는 경우와 아닌 경우의 두 가지 종류가 있는데 이는 추후 Register Slice의 종류에 대해서 이야기 할 때 알아보기로 한다.



Register Slice의 기본형태

단일 채널에 대한 Register Slice를 이용하여 전체 AXI Register Slice를 구성하는 경우의 모양은 아래 그림과 같다. Write address channel(AW), Write data channel(WD), Read address channel은 AXI master가 sender가 되고 AXI slave가 receiver가 되는 반면, Write response channel, Read response channel의 경우는 반대로 AXI slave가 sender가 되고 AXI master가 receiver가 된다.



Register Slice를 다섯개의 channel에 연결하는 방법

당연히 특정 channel에만 Register Slice를 추가하고 싶은 경우에는 그렇게 만들어도 된다. 또한 Register slice를 cascading하여 사용하면 FIFO가 된다. 하지만 그 FIFO는 latency가 depth만큼 생기므로 효율적인 FIFO라고 할 수는 없다.

(*) Register Slice의 종류

Register Slice는 Information과 VALID는 항상 register로 latch되지만 READY신호는 register로 latch되어 전달되는 경우와 아닌 경우 두 가지가 있다.

- Registered forward path

Registered forward path의 경우는 Receiver의 READY 신호가 combinational하게 sender쪽으로 전달된다. 즉 VALID/Information에 대해서는 timing isolation이 이루어지지만 READY신호는 그렇지 않다. Registered forward path의 경우 Information에 대한 register를 하나만 가지면 되기 때문에 gate count가 작다는 장점이 있다.

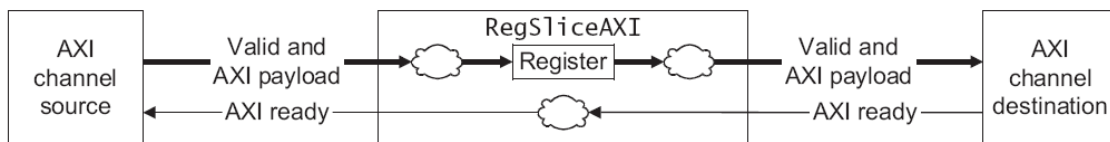


Figure 3 Registered forward path configuration

- Fully registered

Fully registered는 READY도 register로 latch되는 형태이다. 이 경우 sender와 receiver사이에 완벽하게 timing isolation이 된다는 장점이 있다. 하지만 READY가 1 cycle의 delay를 가지며 sender쪽에 전달되기 때문에, READY가 low로 떨어지는 경우 1 cycle동안 sender는 Information을 전달할 수 있다. 따라서 Information을 저장할 register가 depth 2인 que의 형태를 가질 수 밖에 없고 그로 인해 Registered forward path의 형태보다 gate count가 늘어나는 단점이 있다.

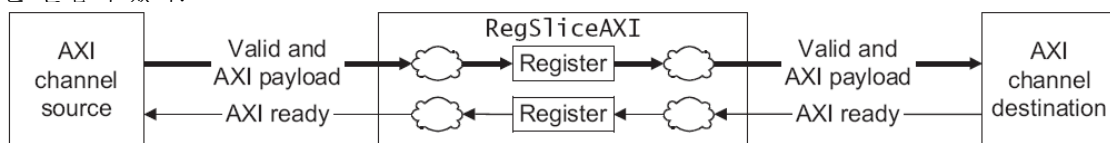
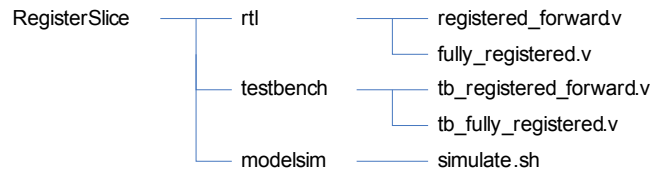


Figure 2 Fully registered configuration

(*) 제공되는 module 및 파일

두 종류의 Register Slice는 각각 registered_forward.v와 fully_registered.v라는 verilog source code로 제공된다. 이 module의 테스트를 위해 각각 tb_registered_forward.v, tb_fully_registered.v라는 test bench verilog source와 simulate.sh이라는 modelsim용 compile&simulation shell script가 제공된다.



Simulation은 random하게 생성된 8bit information을 register slice를 통해서 전달하고 전달된 정보를 원래 information과 비교하여 test를 하게 된다. 단순히 simulate.sh를 수행하면 compile과 simulation이 이루어지게 된다.

(*) Signals

registered_forward.v와 fully_registered.v는 다음과 같은 동일한 입출력 신호를 가지고 있다.

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
ACLK	IN	Clock Generator	AXI clock
ARESETn	IN	Reset Controller	AXI reset
INFORMATION_S[X:0]	IN	Sender	Information such as addr/data/control
VALID_S	IN	Sender	VALID signal from sender
READY_S	OUT	Sender	READY signal to sender
INFORMATION_R[X:0]	OUT	Receiver	Information such as addr/data/control
VALID_R	OUT	Receiver	VALID signal to receiver
READY_R	IN	Receiver	READY signal from receiver

INFORMATION_S와 INFORMATION_R의 bitwidth는 verilog source code의 parameter인 WIDTH의 값으로 정해지므로 사용자는 configuration하여 사용하면 된다.

(*) Integration 방법

Integration은 비교적 간단하다. Read Data Channel에 대해서 Register Slice를 사용하는 예를 살펴보면 다음과 같다. Read Data Channel은 정보의 sender가 AXI slave이고 receiver가 AXI master가 된다. Information으로는 Read Data Channel을 통해서 전달되는 정보 모두를 concatenation하여 연결해 주면 된다. 그 Information의 전체 bit width를 parameter(아래 소스코드상의 width)로 넘겨주면 된다.

```

fully_registered #(width) rd_fully_registered(
    .ACLK (ACLK) ,
    .ARESETn (ARESETn) ,
    .INFORMATION_S ({RID_Slave,          RDATA_Slave,          RRESP_Slave,
RLAST_Slave}),
  
```

```
        .VALID_S(RVALID_Slave),
        .READY_S(RREADY_Slave),
        .INFORMATION_R({RID_Master,      RDATA_Master,      RRESP_Master,
RLAST_Master}),
        .VALID_R(RVALID_Master),
        .READY_R(RREADY_Master)
    );
```
